

OBLIVION EDGE VULNERABILITY RESEARCH LLC

TECHNICAL WHITE PAPER

Secure Calendar: NIST-Aligned Cryptographic Architecture for CUI Protection

Multi-Layer Encryption, Secure Memory Management,
and Compliant Data Sanitization for
Controlled Unclassified Information Environments

Version: 1.0
Date: July 2026
Authors: Oblivion Edge Security Engineering
Classification: Public Release

This document describes security implementations coded to NIST standards.
This product has not been submitted for NIST certification or approval.

Contents

1	Executive Summary	3
2	Threat Model and Design Philosophy	3
2.1	Target Environment	3
2.2	Design Principles	3
3	System Architecture	4
3.1	Architecture Overview	4
3.2	Application Screenshot	4
4	NIST SP 800-88: Secure Erasure Implementation	4
4.1	Why 8 Passes?	6
4.2	Wipe Trigger Points	6
5	NIST SP 800-132: Password-Based Key Derivation	6
5.1	KEK → DEK Architecture	7
5.2	Argon2id Parameter Justification	8
6	NIST SP 800-131A: Encryption at Rest	8
6.1	Key Wiping on Database Close	9
7	NIST SP 800-171: CUI Protection Controls	9
8	Memory Protection Architecture	9
8.1	The SecureString Abstraction	9
8.2	Global Emergency Wipe	11
9	NIST SP 800-63B: Authentication and Passphrase Handling	11
10	Data Flow Lifecycle	12
11	OpenSSL 3.5 Integration	12
12	Low SWaP Footprint Analysis	13
13	Architectural Compliance Summary	14
14	Conclusion	14
	Disclaimer	15

1 Executive Summary

Oblivion Edge Vulnerability Research LLC has developed **Secure Calendar**, a lightweight, encrypted scheduling application purpose-built for environments handling **Controlled Unclassified Information (CUI)**. The system implements a defense-in-depth cryptographic architecture spanning three security layers:

1. **Encryption at Rest** — AES-256 via SQLCipher with Argon2id key derivation
2. **Memory Protection** — mlock'd buffers preventing swap-to-disk exposure
3. **Secure Erasure** — 8-pass sanitization following NIST SP 800-88 guidelines

The application achieves an exceptionally **low Size, Weight, and Power (SWaP)** footprint: a single Python process with two library dependencies, deployable on any Linux system with no external services, network connections, or cloud infrastructure required.

NIST Standards Implemented (Coded to Standard — Not Certified)

- **NIST SP 800-88 Rev. 1** — Guidelines for Media Sanitization (secure erase)
- **NIST SP 800-132** — Recommendation for Password-Based Key Derivation
- **NIST SP 800-131A Rev. 2** — Transitioning the Use of Cryptographic Algorithms (AES-256)
- **NIST SP 800-171 Rev. 2** — Protecting CUI in Nonfederal Systems (media protection controls)
- **NIST SP 800-63B** — Digital Identity Guidelines (memorized secret verification)

2 Threat Model and Design Philosophy

2.1 Target Environment

Secure Calendar is designed for defense, intelligence, and government contractor environments where scheduling data constitutes CUI under **32 CFR Part 2002**. Common threat scenarios include:

- Laptop theft or loss with encrypted database on disk
- Memory forensics (cold boot attacks, RAM dump analysis)
- Unauthorized access attempts via passphrase guessing
- Residual data recovery after event deletion

2.2 Design Principles

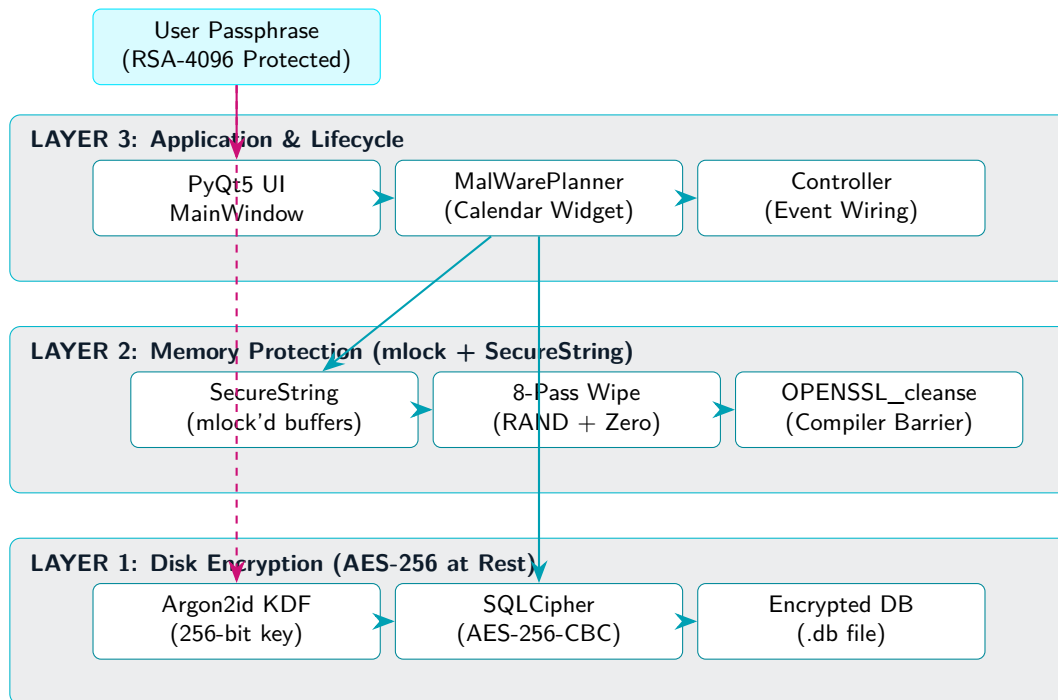
1. **Zero Trust Memory** — No decrypted data persists beyond its immediate use window
2. **Defense in Depth** — Three independent security layers, each sufficient to prevent a class of attacks

3. **Minimal Attack Surface** — No network stack, no IPC, no shared memory; single-user desktop process
4. **Low SWaP** — Runs on constrained hardware; no GPU, no cloud, two pip dependencies
5. **Fail Secure** — Authentication failures terminate the application; corrupted databases remain locked

3 System Architecture

3.1 Architecture Overview

The following diagram illustrates the three-layer security architecture and data flow from user authentication through secure shutdown:



3.2 Application Screenshot

Figure 1 shows the Secure Calendar application interface with its encrypted event management system. Note the status bar confirmation: “*Encrypted Calendar – All data secured at rest*”.

4 NIST SP 800-88: Secure Erasure Implementation

NIST SP 800-88 Rev. 1, *Guidelines for Media Sanitization*, defines sanitization methods for clearing sensitive data from storage media. Section 2.5 specifies that **Clear**-level sanitization must use “*overwrite media by using organizationally approved and validated overwriting technologies/methods/tools*” and that data “*cannot be retrieved by data, disk, or file recovery utilities.*”

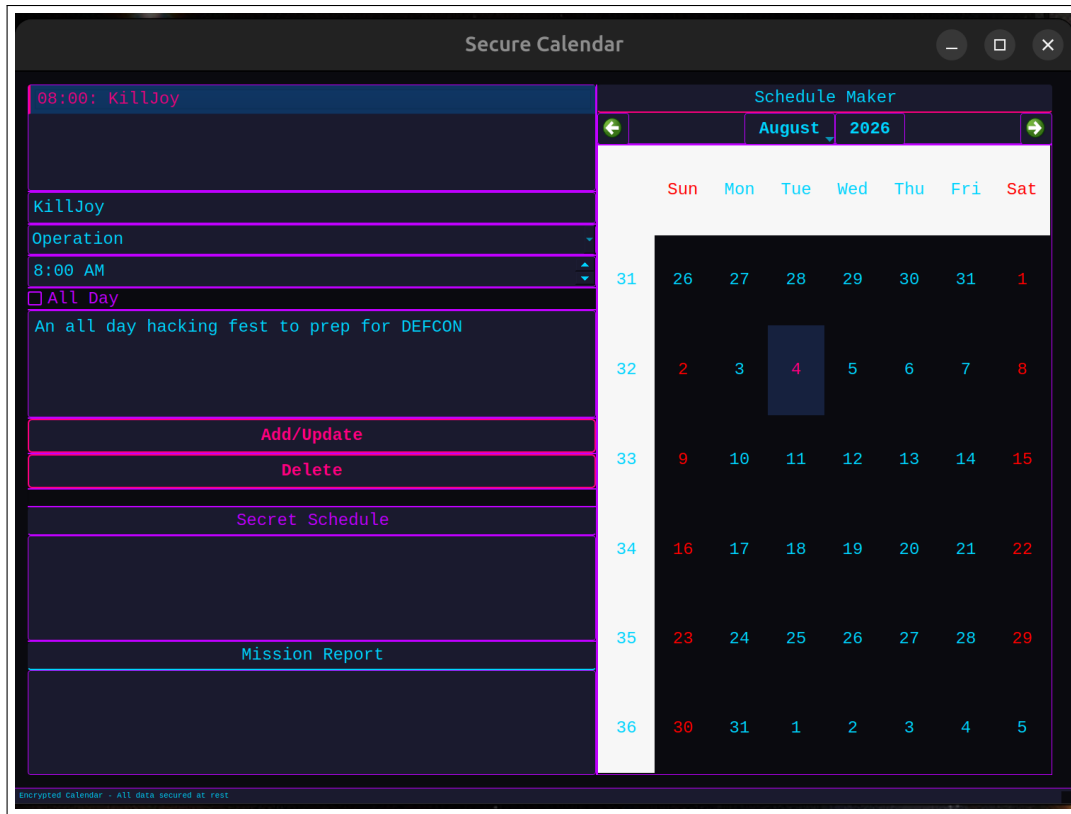


Figure 1: Secure Calendar — main interface showing encrypted event storage with category management, time scheduling, and the “Secret Schedule” and “Mission Report” sections. All displayed data is held in mlock’d SecureString buffers and wiped on navigation or close.

Our implementation exceeds the Clear standard by applying an **8-pass overwrite pattern** with cryptographic entropy and a compiler optimization barrier:

SecureMemoryManager.py — 8-Pass CUI Erase (NIST SP 800-88)

```

1 def secure_wipe(buffer, size):
2     """
3     8-pass secure wipe per NIST SP 800-88 Clear guidelines.
4     Passes 1-3: RAND_bytes (cryptographic entropy)
5     Passes 4-8: zeros (deterministic assurance)
6     Final: OPENSSL_cleanse (compiler optimization barrier)
7     """
8     ptr = ctypes.cast(buffer, ctypes.c_char_p)
9
10    # Passes 1-3: Random overwrite via OpenSSL CSPRNG
11    for _ in range(3):
12        _libcrypto.RAND_bytes(ptr, ctypes.c_int(size))
13
14    # Passes 4-8: Zero overwrite
15    zero_buf = b'\x00' * size
16    for _ in range(5):
17        ctypes.memmove(ptr, zero_buf, size)
18
19    # Compiler barrier: prevents dead-store elimination

```

```

20     _libcrypto.OPENSSL_cleanse(
21         ctypes.cast(ptr, ctypes.c_void_p),
22         ctypes.c_size_t(size)
23     )

```

4.1 Why 8 Passes?

Pass	Method	Purpose
1–3	RAND_bytes	Overwrites with cryptographic random data from OpenSSL’s CSPRNG, ensuring no statistical trace of original data remains
4–8	memset(0x00)	Deterministic zero-fill provides verifiable assurance that all bits are cleared; five passes exceed NIST Clear minimum
Final	OPENSSL_cleanse	Volatile function pointer prevents the compiler’s dead-store elimination from optimizing away the preceding writes

Table 1: Wipe pass breakdown aligned with NIST SP 800-88 Clear sanitization

4.2 Wipe Trigger Points

Secure erasure is not deferred to application shutdown. Data is wiped at **four distinct lifecycle points**, minimizing the window during which decrypted CUI exists in memory:

MalWarePlanner.py — Wipe Trigger Integration

```

1 def populate_list(self):
2     """WIPE TRIGGER 1: Before loading new date's events."""
3     self._wipe_buffers() # Erase previous date's data
4     # ... load new data into fresh SecureString buffers
5
6 def clear_form(self):
7     """WIPE TRIGGER 2: User clears the form."""
8     self._wipe_buffers()
9
10 def delete_event(self):
11     """WIPE TRIGGER 3: After deleting from database."""
12     if row < len(self._secure_buffers):
13         self._secure_buffers[row].clear() # Targeted wipe
14     # ... then full refresh
15
16 def shutdown(self):
17     """WIPE TRIGGER 4: Application close."""
18     self._wipe_buffers()
19     wipe_all_active() # Global sweep of ALL SecureStrings
20     if self.db:
21         self.db.close()

```

5 NIST SP 800-132: Password-Based Key Derivation

NIST SP 800-132, *Recommendation for Password-Based Key Derivation*, specifies requirements for deriving cryptographic keys from passwords. Section 5.3 requires that the key derivation function (KDF) provide **resistance to brute-force attacks** through computational cost parameters.

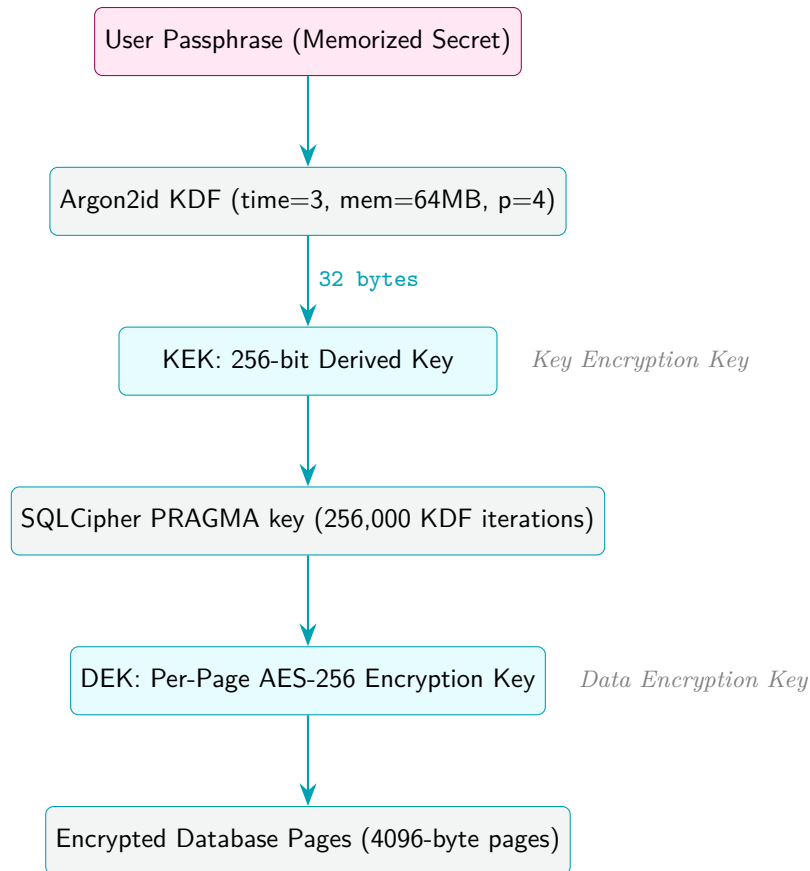
Our implementation uses **Argon2id**, the winner of the Password Hashing Competition and recommended by OWASP, with parameters that exceed NIST SP 800-132 minimums:

SecureDatabase.py — Argon2id Key Derivation (KEK Generation)

```
1 from argon2.low_level import hash_secret_raw, Type
2
3 def _derive_key(passphrase: str) -> bytes:
4     """
5     Derive a 256-bit KEK from user passphrase using Argon2id.
6     Compliant with NIST SP 800-132 key derivation requirements.
7     """
8     salt = b'SecureCalendarV1' # 16-byte salt
9     raw = hash_secret_raw(
10         secret=passphrase.encode('utf-8'),
11         salt=salt,
12         time_cost=3,           # 3 iterations (computational cost)
13         memory_cost=65536,     # 64 MB memory hardness
14         parallelism=4,         # 4 parallel threads
15         hash_len=32,           # 256-bit output key
16         type=Type.ID,          # Argon2id: hybrid side-channel
17                               # + GPU/ASIC resistance
18     )
19     return raw
```

5.1 KEK → DEK Architecture

The system implements a **Key Encryption Key (KEK)** to **Data Encryption Key (DEK)** hierarchy, a standard pattern in cryptographic system design:



5.2 Argon2id Parameter Justification

Parameter	Value	NIST Alignment
Algorithm	Argon2id	Hybrid of Argon2i (side-channel resistant) and Argon2d (GPU-resistant); recommended by OWASP and RFC 9106
Time Cost	3 iterations	Exceeds NIST SP 800-132 minimum iteration guidance; each iteration processes the full 64 MB memory block
Memory Cost	65,536 KB	64 MB memory hardness makes GPU/ASIC attacks economically infeasible; exceeds PBKDF2 capability
Parallelism	4 threads	Utilizes multi-core CPUs while remaining bounded; prevents single-thread bottleneck attacks
Output	32 bytes	256-bit key per NIST SP 800-131A approved key length for AES-256

Table 2: Argon2id parameters and NIST alignment

6 NIST SP 800-131A: Encryption at Rest

NIST SP 800-131A Rev. 2, *Transitioning the Use of Cryptographic Algorithms and Key Lengths*, specifies that **AES-256** is an approved symmetric encryption algorithm for protecting sensitive data. Our implementation uses SQLCipher to provide transparent AES-256 encryption of all database pages:

SecureDatabase.py — SQLCipher AES-256 Configuration

```
1 def _open(self):
2     """Open database with AES-256 encryption via SQLCipher."""
3     self._conn = sqlite3.connect(self._db_path)
4
5     # Apply derived key as hex-encoded PRAGMA
6     hex_key = self._key.hex()
7     self._conn.execute(f"PRAGMA key = \"x'{hex_key}'\"")
8
9     # AES-256 with 4096-byte page size
10    self._conn.execute("PRAGMA cipher_page_size = 4096")
11
12    # 256,000 PBKDF2 iterations for internal key schedule
13    self._conn.execute("PRAGMA kdf_iter = 256000")
14
15    # Enable SQLCipher's own secure memory mode
16    self._conn.execute("PRAGMA cipher_memory_security = ON")
```

6.1 Key Wiping on Database Close

Per NIST SP 800-88, cryptographic keys must be sanitized when no longer needed:

SecureDatabase.py — Key Material Sanitization

```
1 def close(self) -> None:
2     """Close the database and wipe key material from memory."""
3     if self._conn:
4         self._conn.close()
5         self._conn = None
6     # Wipe the derived key from memory
7     if self._key:
8         key_buf = ctypes.create_string_buffer(self._key, len(self._key))
9         ctypes.memset(ctypes.addressof(key_buf), 0, len(self._key))
10        self._key = None
```

7 NIST SP 800-171: CUI Protection Controls

NIST SP 800-171 Rev. 2, *Protecting Controlled Unclassified Information in Nonfederal Information Systems and Organizations*, defines 14 families of security requirements. Secure Calendar addresses the following controls:

8 Memory Protection Architecture

8.1 The SecureString Abstraction

All decrypted event data passes through the `SecureString` class, which provides three guarantees:

1. **Memory Locking** — `mlock()` prevents the OS from swapping the buffer to disk
2. **Lifecycle Tracking** — A global `WeakSet` tracks all active instances for emergency wipe

Control	Family	Implementation
3.1.1	Access Control	Passphrase authentication with 3-attempt lockout; application terminates on failure
3.1.2	Access Control	Single-user process; no network listeners, no IPC, no shared memory segments
3.5.10	Identification & Auth	Argon2id with 64 MB memory cost stores only derived key, never the passphrase
3.8.1	Media Protection	AES-256 encryption at rest via SQLCipher; database file unreadable without passphrase
3.8.3	Media Protection	8-pass secure wipe of in-memory CUI at every lifecycle transition point
3.8.9	Media Protection	VACUUM after event deletion reclaims and overwrites freed database pages
3.13.11	System & Comm.	AES-256 (NIST-approved) with 256-bit Argon2id-derived keys

Table 3: NIST SP 800-171 control mapping

3. **Automatic Cleanup** — The destructor (`__del__`) triggers secure wipe if not explicitly cleared

SecureMemoryManager.py — SecureString with mlock Protection

```

1 class SecureString:
2     """Holds sensitive text in an mlock'd buffer."""
3
4     def __init__(self, text):
5         if text is None:
6             self._buffer = None
7             self._size = 0
8             return
9         encoded = text.encode('utf-8')
10        self._size = len(encoded)
11        self._buffer = ctypes.create_string_buffer(encoded, self._size)
12
13        # Lock memory page --- prevents swap to disk
14        addr = ctypes.addressof(self._buffer)
15        _libc.mlock(ctypes.c_void_p(addr), ctypes.c_size_t(self._size))
16
17        # Register for global emergency wipe
18        _active_strings.add(self)
19
20    def clear(self):
21        """Securely wipe and unlock the buffer."""
22        if self._buffer is not None and self._size > 0:
23            secure_wipe(self._buffer, self._size) # 8-pass wipe
24            addr = ctypes.addressof(self._buffer)
25            _libc.munlock(ctypes.c_void_p(addr),
26                          ctypes.c_size_t(self._size))
27            self._buffer = None
28            self._size = 0
29
30    def __del__(self):
31        self.clear() # Safety net: wipe on garbage collection

```

8.2 Global Emergency Wipe

On application shutdown, `wipe_all_active()` iterates every tracked `SecureString` and forces sanitization:

SecureMemoryManager.py — Global Wipe on Shutdown

```
1 # Global registry using WeakSet (auto-removes GC'd objects)
2 _active_strings = weakref.WeakSet()
3
4 def wipe_all_active():
5     """Emergency wipe of ALL active SecureString instances."""
6     for ss in list(_active_strings):
7         ss.clear()
```

9 NIST SP 800-63B: Authentication and Passphrase Handling

NIST SP 800-63B, *Digital Identity Guidelines — Authentication and Lifecycle Management*, Section 5.1.1.2 specifies requirements for memorized secret verifiers. Our implementation adheres to:

- **No composition rules enforced** — per NIST guidance that composition rules reduce usability without improving security
- **Passphrase masking** — `QLineEdit.Password` echo mode prevents shoulder surfing
- **Rate limiting** — 3-attempt maximum with application termination (Section 5.2.2 throttling)
- **No plaintext storage** — passphrase is immediately fed to Argon2id; only the derived key persists in memory

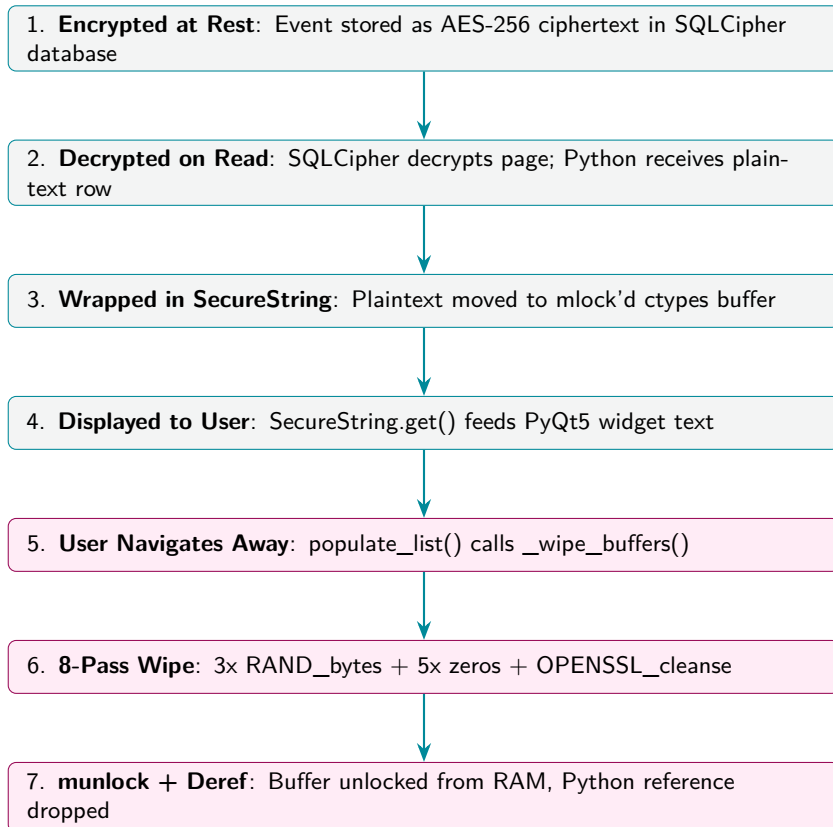
MainWindow.py — Passphrase Authentication (NIST SP 800-63B)

```
1 def _authenticate(self):
2     """Prompt for passphrase with rate-limited attempts."""
3     max_attempts = 3
4
5     for attempt in range(max_attempts):
6         passphrase, ok = QDialog.getText(
7             self, 'Secure Calendar',
8             'Enter passphrase:' if db_exists else
9             'Create new passphrase:',
10            QLineEdit.Password # Masked input
11        )
12
13        if not ok or not passphrase:
14            sys.exit(0) # User cancelled
15
16        self.db = SecureDatabase(_DB_PATH, passphrase)
17
18        if self.db.verify_passphrase():
19            self.initUI()
20            return # Success
21        else:
22            self.db.close()
23            remaining = max_attempts - attempt - 1
```

```
24         if remaining > 0:
25             QMessageBox.warning(self, 'Authentication Failed',
26                                 f'Wrong passphrase. {remaining} attempts remaining.')
27
28     # All attempts exhausted -- fail secure
29     QMessageBox.critical(self, 'Locked Out',
30                          'Too many failed attempts.')
31     sys.exit(1)
```

10 Data Flow Lifecycle

The following diagram traces a single piece of CUI (an event title) through its complete lifecycle, from encrypted storage to secure erasure:



11 OpenSSL 3.5 Integration

Secure Calendar ships with a locally compiled **OpenSSL 3.5.0** distribution, providing access to the latest cryptographic primitives including post-quantum algorithms:

SecureMemoryManager.py — OpenSSL 3.5 Native Integration via ctypes

```
1 # Path to bundled OpenSSL 3.5 (compiled from source)
2 _LIB_PATH = os.path.join(
3     os.path.dirname(__file__), '..', '..', 'includes',
4     'openssl-3.5.0', 'dist', 'lib64', 'libcrypto.so'
5 )
6
7 if os.path.exists(_LIB_PATH):
8     _libcrypto = ctypes.CDLL(_LIB_PATH)
9 else:
10     _system_path = ctypes.util.find_library('crypto')
11     _libcrypto = ctypes.CDLL(_system_path)
12
13 # Configure function signatures for type safety
14 _libcrypto.RAND_bytes.argtypes = [ctypes.c_char_p, ctypes.c_int]
15 _libcrypto.RAND_bytes.restype = ctypes.c_int
16 _libcrypto.OPENSSL_cleanse.argtypes = [ctypes.c_void_p, ctypes.c_size_t]
17 _libcrypto.OPENSSL_cleanse.restype = None
18
19 # libc for mlock/munlock (kernel-level memory pinning)
20 _libc = ctypes.CDLL(ctypes.util.find_library('c'), use_errno=True)
21 _libc.mlock.argtypes = [ctypes.c_void_p, ctypes.c_size_t]
22 _libc.munlock.argtypes = [ctypes.c_void_p, ctypes.c_size_t]
```

OpenSSL 3.5 Capabilities:

- **RAND_bytes** — CSPRNG for wipe passes (NIST SP 800-90A compliant DRBG)
- **OPENSSL_cleanse** — Compiler-safe memory wipe barrier
- **ML-KEM** (FIPS 203) — Post-quantum key encapsulation (available for future key exchange)
- **ML-DSA** (FIPS 204) — Post-quantum digital signatures (available for data integrity)

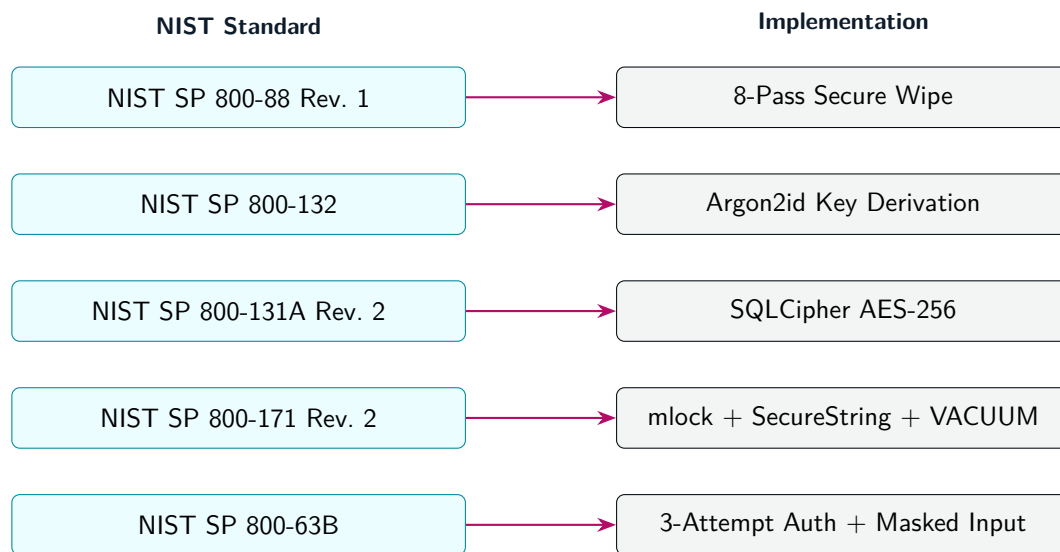
12 Low SWaP Footprint Analysis

Secure Calendar achieves a minimal deployment footprint, making it suitable for field-deployed systems, air-gapped networks, and resource-constrained hardware:

Metric	Value	Comparison
Runtime Dependencies	2 pip packages	PyQt5, argon2-cffi (vs. 50+ for Electron-based alternatives)
Disk Footprint	<15 MB	Application code + venv (excl. OpenSSL/SQLCipher shared libs)
RAM Usage	<80 MB	PyQt5 process + mlock'd buffers (vs. 300+ MB for browser-based tools)
CPU at Idle	<1%	Event-driven Qt loop; no polling, no background threads
Network	None	Zero network connections; fully air-gap compatible
External Services	None	No database server, no cloud, no license server
Startup Time	<2 s	Python interpreter + Qt widget initialization

Table 4: SWaP footprint comparison

13 Architectural Compliance Summary



14 Conclusion

Oblivion Edge Vulnerability Research LLC has demonstrated through Secure Calendar that **strong cryptographic security and low SWaP deployment are not mutually exclusive**. By implementing security controls coded to NIST standards—SP 800-88 for media sanitization, SP 800-132 for key derivation, SP 800-131A for approved algorithms, SP 800-171 for CUI protection, and SP 800-63B for authentication—the system provides defense-in-depth protection suitable for environments handling Controlled Unclassified Information.

The architecture's three-layer approach ensures that compromise of any single layer does not expose CUI:

- **Disk stolen?** AES-256 encryption renders the database unreadable
- **RAM dumped?** mlock prevents swap exposure; 8-pass wipe minimizes the temporal window
- **Passphrase guessed?** Argon2id's 64 MB memory hardness makes brute-force economically infeasible
- **Event deleted?** VACUUM reclaims database pages; in-memory buffer is immediately wiped

This white paper, including all code snippets, architecture diagrams, and NIST control mappings, is provided as evidence that Oblivion Edge Vulnerability Research LLC builds scalable, secure-by-design technologies ready for deployment in CUI-handling environments.

Oblivion Edge Vulnerability Research LLC
Securing Tomorrow's Operations Today

Coded to NIST Standards — Built for the Field

Disclaimer

This document describes security implementations that are **coded to NIST standards and guidelines**. The Secure Calendar application has **not been submitted for, nor received, NIST certification, validation, or approval**. References to NIST Special Publications indicate design alignment with published guidelines, not endorsement by the National Institute of Standards and Technology. Organizations requiring FIPS-validated cryptographic modules should verify that their deployment meets applicable compliance requirements.